

บทที่ 5 การเตรียมข้อมูล

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมาย ความสำคัญ และกระบวนการเตรียมข้อมูล (Data Preprocessing) ในงานเหมืองข้อมูล รวมถึงระบุแพ็คเกจ Python ที่เกี่ยวข้อง ได้แก่ Pandas, NumPy, Scikit-Learn, NLTK และ SciPy
2. จัดการกับค่าสูญหาย (Missing Values) โดยใช้วิธีต่างๆ ได้แก่ การลบ (Dropping) การแทนด้วยค่าคงที่ ค่าเฉลี่ย มัธยฐาน ฐานนิยม และ Forward/Backward Fill รวมถึง Interpolation สำหรับข้อมูล Time Series
3. ตรวจจับและลบค่าผิดปกติ (Outliers) โดยใช้วิธี IQR (Interquartile Range) และวิธีทางสถิติ Z-Score โดยอ้างอิงหลักการ Empirical Rule (68-95-99.7 Rule)
4. ลดขนาดข้อมูล (Data Reduction) ด้วยการตัดมิติ (Dimension Elimination) และการสุ่มตัวอย่าง (Sampling) ในรูปแบบต่างๆ ได้แก่ Simple Random Sampling, Stratified Sampling และ Bootstrap Sampling
5. แปลงข้อมูลต่อเนื่องเป็นช่วงหมวดหมู่ (Data Discretization) ด้วยวิธี Equal Width Binning, Equal Frequency Binning และเงื่อนไขหลายข้อ รวมถึงปรับสเกลข้อมูล (Data Scaling) ด้วยวิธี Min-Max Normalization, Z-Score Standardization และ Decimal Scaling
6. สร้างและสืบค้นข้อมูลจาก Data Cube แบบหลายมิติด้วย Atoti และวิเคราะห์ข้อมูลด้วย Pivot Table ผ่าน Pandas เพื่อสนับสนุนการวิเคราะห์แบบ OLAP ในงาน Data Warehouse

5.1 บทนำ

การเตรียมข้อมูล (Data Preprocessing) คือกระบวนการทำความสะอาด แปลงรูป และเตรียมข้อมูลสำหรับการวิเคราะห์ เป็นขั้นตอนสำคัญในกระบวนการทำเหมืองข้อมูล เพราะช่วยให้มั่นใจว่าข้อมูลอยู่ในรูปแบบที่สามารถวิเคราะห์ได้อย่างง่ายและแม่นยำ

Python มีแพ็คเกจที่ใช้กันทั่วไปสำหรับการเตรียมข้อมูล ได้แก่:

7. Pandas: ไลบรารีสำหรับทำงานกับข้อมูลในรูปแบบตาราง มีเครื่องมือหลากหลายสำหรับอ่าน เขียน และจัดการข้อมูล รวมถึงการจัดการค่าสูญหาย
8. NumPy: ไลบรารีสำหรับทำงานกับอาร์เรย์และเมทริกซ์ของข้อมูลตัวเลข มีฟังก์ชันทางคณิตศาสตร์และสถิติที่หลากหลาย
9. Scikit-learn: ไลบรารีสำหรับ Machine Learning ใน Python มีเครื่องมือสำหรับเตรียมข้อมูล เช่น Feature Scaling, Normalization และ One-Hot Encoding
10. NLTK: ไลบรารีสำหรับการประมวลผลภาษาธรรมชาติ มีเครื่องมือสำหรับการเตรียมข้อความ เช่น Tokenization, Stemming และ Lemmatization
11. SciPy: ไลบรารีสำหรับการคำนวณทางวิทยาศาสตร์ มีเครื่องมือสำหรับการเตรียมข้อมูล เช่น Interpolation และ Smoothing

5.2 แนวคิดและทฤษฎีที่เกี่ยวข้อง

5.2.1 ความสำคัญของการเตรียมข้อมูลในกระบวนการเหมืองข้อมูล

การเตรียมข้อมูล (Data Preprocessing) เป็นขั้นตอนที่ใช้เวลามากที่สุด ในกระบวนการทำเหมืองข้อมูล โดยทั่วไป นักวิทยาศาสตร์ข้อมูลใช้เวลาถึง 60–80% ของโครงการทั้งหมดไปกับการเตรียมข้อมูล Tan, Steinbach และ Kumar (2006)

ระบุว่ากระบวนการเตรียมข้อมูลครอบคลุมเทคนิคหลักหลายกลุ่มที่มีความสัมพันธ์กัน ได้แก่ การรวมข้อมูล (Aggregation) การสุ่มตัวอย่าง (Sampling) การลดมิติ (Dimensionality Reduction) การคัดเลือก features (Feature Subset Selection) การสร้าง features ใหม่ (Feature Creation) การแยกช่วง (Discretization and Binarization) และการแปลงตัวแปร (Variable Transformation) เป้าหมายร่วมของทุกเทคนิคคือการปรับปรุงคุณภาพของข้อมูลให้เหมาะสมกับการวิเคราะห์ทั้งในด้านเวลา ต้นทุน และความแม่นยำ

5.2.2 ทฤษฎีการจัดการค่าที่หายไป (Missing Values)

ค่าสูญหายเกิดขึ้นได้จากหลายสาเหตุ เช่น ความผิดพลาดในการเก็บข้อมูล อุปกรณ์วัดขัดข้อง หรือข้อมูลที่ไม่เกี่ยวข้องกับ object นั้น แนวทางการจัดการค่าสูญหายตามที่ Tan et al. (2006) นำเสนอมีสามวิธีหลัก ได้แก่ (1) การลบ objects หรือ attributes ที่มีค่าหายไป (Elimination) ซึ่งเหมาะสมเมื่อมีค่าหายไปเพียงเล็กน้อย แต่ต้องระวังว่า attributes ที่ถูกลบอาจเป็นข้อมูลสำคัญต่อการวิเคราะห์ (2) การประมาณค่าสูญหาย (Imputation) ซึ่งสำหรับข้อมูลต่อเนื่องอาจใช้ค่าเฉลี่ยของ nearest neighbors หรือการ interpolation จากจุดข้อมูลรอบข้าง ส่วนข้อมูลเชิงหมวดหมู่ใช้ค่าที่พบบ่อยที่สุด (Mode) และ (3) การละเว้นค่าสูญหายระหว่างการวิเคราะห์ (Ignore) โดยอัลกอริทึมบางชนิดสามารถคำนวณโดยใช้เฉพาะ attributes ที่มีค่าเพื่อหลีกเลี่ยงการใช้ค่าสูญหาย

5.2.3 ทฤษฎีค่าผิดปกติ (Outliers) และการตรวจจับ

ค่าผิดปกติ (Outliers) คือ data objects ที่มีลักษณะแตกต่างอย่างมีนัยสำคัญจาก objects ส่วนใหญ่ในชุดข้อมูล ในบางกรณีค่าผิดปกติถือเป็น noise ที่ต้องกำจัด แต่ในบางบริบทกลับเป็นข้อมูลที่มีคุณค่าที่สุด เช่น การตรวจจับการฉ้อโกงหรือความผิดปกติในระบบ วิธีการตรวจจับค่าผิดปกติที่นิยมใช้มีสองแนวทางหลัก

วิธี IQR (Interquartile Range) เป็นวิธีที่ทนทานต่อค่าผิดปกติสูงเนื่องจากอิงกับ percentile ไม่ใช่ค่าเฉลี่ย คำนวณโดยหา $IQR = Q3 - Q1$ จากนั้นกำหนดขอบเขตล่าง = $Q1 - 1.5 \times IQR$ และขอบเขตบน = $Q3 + 1.5 \times IQR$ ค่าที่อยู่นอกขอบเขตเหล่านี้ถือว่าเป็น Outlier ค่าคงที่ 1.5 เป็นมาตรฐานที่ John Tukey นำเสนอ ซึ่งใช้ในการวาด Box Plot โดย whiskers ยาว $1.5 \times IQR$ จาก $Q1$ และ $Q3$

$$IQR = Q3 - Q1$$

$$\text{Lower Bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q3 + 1.5 \times IQR$$

วิธี Z-Score (Statistical Method) อิงบนสมมติฐานว่าข้อมูลมีการกระจายแบบ

± 3 ส่วนเบี่ยงเบนมาตรฐาน ซึ่งเรียกว่า Empirical Rule หรือ 68-95-99.7 Rule นั่นข้อมูลที่อยู่นอกช่วง $\text{mean} \pm 3\sigma$ จึงถือว่าเป็น Outlier เนื่องจากความน่าจะเป็นที่จะเกิดขึ้นโดยบังเอิญมีเพียง 0.3% ข้อจำกัดของวิธีนี้คือต้องการให้ข้อมูลมีการกระจายปกติ และอ่อนไหวต่อ Outlier เองเนื่องจากใช้ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานในการคำนวณ

5.2.4 ทฤษฎีการลดมิติและการคัดเลือก Features

การลดมิติ (Dimensionality Reduction) มีความจำเป็นเนื่องจากปัญหา “Curse of Dimensionality” ซึ่งหมายถึงปัญหาที่ข้อมูลที่มี attributes จำนวนมากทำให้อัลกอริทึมทำงานได้แย่ลง

โดยปริภูมิของข้อมูลจะกระจายและค่าความคล้ายคลึงกันระหว่าง data objects สูญเสียความหมาย Tan et al. (2006) แบ่งวิธีการลดมิติออกเป็นสองแนวทางหลัก

การลดมิติด้วย Linear Algebra วิธีที่แพร่หลายที่สุดคือ Principal Components Analysis (PCA) ซึ่งหา attributes ใหม่ (principal components) ที่เป็น linear combinations ของ attributes เดิม โดยมีสมบัติว่า (1) แต่ละ component ตั้งฉากกัน (Orthogonal) (2) component แรกอธิบายความแปรปรวนของข้อมูลได้มากที่สุด และ (3) components ต่อๆ มาอธิบายความแปรปรวนที่เหลือในลำดับที่ลดลง โดยใช้เพียงไม่กี่

principal

components

แรกก็สามารถอธิบายข้อมูลได้เป็นส่วนใหญ่โดยสูญเสียข้อมูลน้อยที่สุด เทคนิคที่เกี่ยวข้องอีกชนิดคือ Singular Value Decomposition (SVD) ซึ่งมีความสัมพันธ์ใกล้ชิดกับ PCA

การคัดเลือก Feature Subset (Feature Subset Selection)

เป็นอีกวิธีในการลดมิติโดยการเลือกเฉพาะ attributes ที่มีประโยชน์จริงๆ ออกมาใช้ Tan et al. (2006) ชี้ว่า attributes สามารถแบ่งเป็น Redundant features ที่ซ้ำซ้อนกับ attributes อื่น และ Irrelevant features ที่ไม่มีข้อมูลเป็นประโยชน์ต่องานที่ทำ ทั้งสองประเภทควรถูกกำจัดออก แนวทางในการเลือก Feature Subset มีสามแบบ ได้แก่ Embedded Approach ซึ่งอัลกอริทึมตัดสินใจเองระหว่างการทำงาน (เช่น Decision Tree), Filter Approach ซึ่งคัดเลือก features ก่อนรันอัลกอริทึมโดยใช้เกณฑ์อิสระ เช่น ค่า correlation ที่ต่ำ และ Wrapper Approach ซึ่งใช้อัลกอริทึมเป้าหมายเป็น black box เพื่อประเมิน subset ต่างๆ

5.2.5 ทฤษฎีการสุ่มตัวอย่าง (Sampling)

การสุ่มตัวอย่าง (Sampling) เป็นเทคนิคสำคัญในการลดขนาดข้อมูล โดยเลือก subset ของ data objects ที่เป็นตัวแทนของชุดข้อมูลทั้งหมด Tan et al. (2006) แบ่งประเภทของการสุ่มตัวอย่างออกเป็นสี่แบบหลัก

Simple Random Sampling เป็นการสุ่มแบบง่ายที่สุด โดยทุก object มีความน่าจะเป็นเท่ากันในการถูกเลือก มีสองรูปแบบย่อย ได้แก่ Sampling without Replacement ที่แต่ละ object ถูกเลือกได้เพียงครั้งเดียว และ Sampling with Replacement ที่ object เดิมอาจถูกเลือกซ้ำได้ ซึ่งเป็นพื้นฐานของ Bootstrap Sampling ที่ใช้ในการประเมินความไม่แน่นอนของโมเดล

Stratified Sampling เหมาะสำหรับข้อมูลที่มีกลุ่มย่อย (Strata) ที่มีจำนวนต่างกันมาก (Imbalanced Data) เช่น ข้อมูลโรคที่หายากซึ่งมีผู้ป่วยจำนวนน้อย Simple Random Sampling จะสุ่มได้ตัวอย่างจากกลุ่มเล็กน้อยมาก

ทำให้โมเดลสร้างจากตัวอย่างนั้นมีประสิทธิภาพต่ำสำหรับกลุ่มที่หายาก Stratified Sampling แก้ไขปัญหานี้โดยสุ่มจากแต่ละกลุ่มในสัดส่วนที่กำหนดไว้ล่วงหน้า

ประเด็นสำคัญในการเลือกขนาดตัวอย่างคือตัวอย่างขนาดใหญ่เพิ่มความน่าจะเป็นที่จะเป็นตัวแทนของประชากร แต่ก็มีต้นทุนได้เปรียบด้านการลดขนาดข้อมูล ขณะที่ตัวอย่างขนาดเล็กอาจขาดรูปแบบสำคัญหรือพบรูปแบบที่ไม่มีอยู่จริง การเลือกขนาดตัวอย่างที่เหมาะสมจึงต้องพิจารณาทั้งสองปัจจัยนี้ร่วมกัน

5.2.6 ทฤษฎีการแยกช่วงข้อมูล (Discretization)

การแยกช่วง (Discretization) คือการแปลง continuous attribute ให้เป็น categorical attribute โดยมีขั้นตอนสำคัญสองส่วน ส่วนแรกคือการตัดสินใจว่าจะแบ่งเป็นกี่ช่วง (n intervals) และส่วนที่สองคือการกำหนด split points ที่แบ่งช่วงเหล่านั้น โดย Tan et al. (2006) ระบุว่า ผลลัพธ์ของการแยกช่วงสามารถแทนได้ด้วยชุดของ intervals $\{(x_0, x_1], (x_1, x_2], \dots, (x_{n-1}, x_n)\}$ ซึ่ง x_0 และ x_n อาจเป็น $-\infty$ และ $+\infty$ ตามลำดับ

วิธี Unsupervised Discretization ที่นิยมใช้มีสองแบบ ได้แก่ Equal Width Binning (เท่ากันด้านกว้าง) ซึ่งแบ่งช่วงค่าทั้งหมดออกเป็น n ช่วงที่มีความกว้างเท่ากัน วิธีนี้รีเบียง่ายแต่อ่อนแอต่อ Outlier เพราะค่าผิดปกติจะทำให้ช่วงกว้างมากและข้อมูลส่วนใหญ่กองอยู่ในช่วงเดียว และ Equal Frequency Binning (เท่ากันด้านความถี่ หรือ Equal Depth) ซึ่งพยายามใส่จำนวน objects เท่ากันในแต่ละช่วง วิธีนี้ทนทานต่อ Outlier กว่า เนื่องจาก split points จะถูกกำหนดโดย percentile ของข้อมูล

5.2.7 ทฤษฎีการแปลงตัวแปร (Variable Transformation)

และการปรับสเกล

การแปลงตัวแปร (Variable Transformation) คือการนำฟังก์ชันทางคณิตศาสตร์มาใช้กับค่าของตัวแปรทุกค่า Tan et al. (2006) แบ่งออกเป็นสองประเภทหลัก ได้แก่ Simple Functional Transformations

ซึ่งใช้ฟังก์ชันง่ายๆ เช่น x^k , $\log x$, e^x , $\sqrt{x}$, $1/x$ เพื่อเปลี่ยนรูปแบบการกระจายของข้อมูล ตัวอย่างเช่น $\log$ transformation มักใช้กับข้อมูลที่มีช่วงค่ากว้างมาก เพื่อบีบค่าที่สูงมากเข้ามาใกล้กัน และ Normalization/Standardization ซึ่งเป็นการปรับสเกลข้อมูลให้อยู่ในช่วงที่ต้องการ โดยมีวิธีหลักดังนี้

Min-Max Normalization แปลงข้อมูลให้อยู่ในช่วง $[0, 1]$ โดยสูตร $v' = (v - \min) / (\max - \min)$ ข้อดีคือรักษาความสัมพันธ์สัดส่วนระหว่างค่าต่างๆ ไว้ ข้อเสียคืออ่อนไหวต่อ Outlier มาก เนื่องจาก Outlier เพียงค่าเดียวสามารถทำให้ข้อมูลส่วนใหญ่ถูกบีบอัดให้อยู่ในช่วงแคบ

$$v' = \frac{v - \min}{\max - \min}$$

Z-Score Standardization ใช้สูตร $v' = (v - \bar{x}) / s_x$ ซึ่งสร้างตัวแปรใหม่ที่มีค่าเฉลี่ย = 0 และส่วนเบี่ยงเบนมาตรฐาน = 1 Tan et al. (2006) ระบุว่า การ Standardization มีความจำเป็นอย่างยิ่งเมื่อต้องรวม attributes หลายตัวเข้าด้วยกัน เพื่อป้องกันไม่ให้ตัวแปรที่มีขนาดใหญ่กว่า (เช่น รายได้หน่วยพันดอลลาร์) มีอิทธิพลเหนือตัวแปรที่มีขนาดเล็กกว่า (เช่น อายุหน่วยปี) โดยไม่มีเหตุผลทางสถิติ

$$v' = \frac{v - \bar{X}}{s_x}$$

Decimal Scaling ปรับสเกลด้วยการหารด้วย 10^t เมื่อ t คือจำนวนหลักของค่าสูงสุด ทำให้ค่าสูงสุดที่ได้อยู่ในช่วง $[0, 1)$ วิธีนี้เรียบง่ายและรักษาการกระจายเดิมไว้ได้ แต่ขนาดสเกลอาจไม่สม่ำเสมอหากข้อมูลมีความกว้างของช่วงค่าต่างกันมาก

$$v' = \frac{v}{10^t}$$

ตารางที่ 5.1 สรุปเปรียบเทียบเทคนิคการปรับสเกลทั้งสามวิธี

วิธี	สูตร	ช่วงผลลัพธ์	ความทนทานต่อ Outlier
------	------	-------------	----------------------

Min-Max Normalization	$v' = \frac{v - \min}{\max - \min}$	[0, 1]	ต่ำ (ไวต่อ Outlier)
Z-Score Standardization	$v' = \frac{v - \bar{X}}{s_x}$	mean=0, std=1	ปานกลาง
Decimal Scaling	$v' = \frac{v}{10^t}$ (t = จำนวนหลักของ max)	$ v' < 1$	ปานกลาง

ที่มา: ปรับปรุงจาก Tan, Steinbach, and Kumar (2006), *Introduction to Data Mining*, Chapter 2

5.3 การจัดการค่าที่หายไป

ค่าสูญหาย (Missing Values) เป็นความท้าทายที่พบบ่อยในการวิเคราะห์ข้อมูล ซึ่งต้องได้รับการจัดการเพื่อให้มั่นใจในความถูกต้องและความน่าเชื่อถือของผลลัพธ์ แนวทางการจัดการค่าสูญหายด้วย Pandas มีดังนี้:

12. การลบค่าสูญหาย (Dropping): ใช้วิธี `dropna()` เพื่อลบแถวหรือคอลัมน์ที่มีค่าหายไป เหมาะเมื่อข้อมูลที่หายไปไม่มีข้อมูลสำคัญ
13. การแทนด้วยค่าคงที่ (Constant): ใช้วิธี `fillna()` เพื่อแทนค่าสูญหายด้วยค่าคงที่ เช่น 0 หรือสัญลักษณ์แทน
14. การแทนด้วยค่าแนวโน้มสู่ส่วนกลาง: แทนค่าสูญหายด้วยค่าเฉลี่ย (mean) มัธยฐาน (median) หรือฐานนิยม (mode) ของคอลัมน์นั้น
15. Forward/Backward Fill: ใช้ `fillna(method='ffill')` หรือ `fillna(method='bfill')` เพื่อเติมค่าจากข้อมูลก่อนหน้าหรือถัดไป เหมาะสำหรับข้อมูล Time Series
16. Interpolation: Pandas มีวิธี Interpolation เช่น Linear หรือ Polynomial เพื่อประมาณค่าสูญหายจากจุดข้อมูลรอบข้าง

5.3.1 แบบฝึกหัด – การจัดการค่าที่หายไป

5.7.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/Economy_of_US_na.csv')
```

5.3.1.2 ตรวจจับและรายงานค่าที่หายไป

ใช้ `isnull()` เพื่อตรวจสอบว่าแต่ละเซลล์มีค่าหายไปหรือไม่ และนับจำนวนค่าสูญหายในแต่ละคอลัมน์:

```
df.isnull()
for c in df.columns:
    miss = df[c].isnull().sum()
    print('{} has {} missing value(s)'.format(c, miss))
```

ผลลัพธ์: Year มี 1 ค่าหายไป, GDP_Nominal มี 4 ค่าหายไป, GDP_Growth มี 7 ค่าหายไป

5.3.1.3 การลบค่าที่หายไป

```
df2 = df.dropna() # ลบแถวที่มีค่าหายไปทั้งหมด
df2 = df.dropna(axis=1) # ลบคอลัมน์ที่มีค่าหายไป
df2 = df.dropna(thresh=3) # ลบแถวที่มีค่าไม่หายไปอย่างน้อย 3 ค่า
df2 = df.dropna(thresh=2) # ลบแถวที่มีค่าไม่หายไปอย่างน้อย 2 ค่า
df2 = df.dropna(axis=1, thresh=7) #
ลบคอลัมน์ที่มีค่าไม่หายไปอย่างน้อย 7 ค่า
```

5.3.1.4 การแทนด้วยค่าคงที่

```
df2 = df.fillna('NA') # แทนทุกคอลัมน์ด้วย 'NA'
df['Year_filled'] = df['Year'].fillna('YEAR') #
แทนเฉพาะคอลัมน์ Year
```

5.3.1.5 การแทนด้วย Forward Fill (ffill)

นำค่าจากแถวก่อนหน้ามาเติมแทนค่าสูญหาย:

```
df['GDP_filled_ffill'] =  
df['GDP_Nominal'].fillna(method='ffill')
```

5.3.1.6 การแทนด้วย Backward Fill (bfill)

นำค่าจากแถวถัดไปมาเติมแทนค่าสูญหาย:

```
df['GDP_filled_bfill'] =  
df['GDP_Nominal'].fillna(method='bfill')
```

5.3.1.7 การแทนด้วยค่าเฉลี่ย (Mean)

```
df['GDP_Nominal_filled_mean'] =  
df['GDP_Nominal'].fillna(df['GDP_Nominal'].mean()  
( ))
```

5.3.1.8 การแทนด้วยฐานนิยม (Mode)

```
df['GDP_Nominal_filled_mode'] =  
df['GDP_Nominal'].fillna(df['GDP_Nominal'].mode()  
( ) [0])
```

5.3.1.9 สรุปการเปรียบเทียบวิธีต่างๆ

เปรียบเทียบผลของทุกวิธีสำหรับคอลัมน์ GDP_Growth:

```
df['GDP_Growth_fill_NA'] =  
df['GDP_Growth'].fillna('NA')  
  
df['GDP_Growth_fill_0'] =  
df['GDP_Growth'].fillna(0)  
  
df['GDP_Growth_fill_ffill'] =  
df['GDP_Growth'].fillna(method='ffill')  
  
df['GDP_Growth_fill_bfill'] =  
df['GDP_Growth'].fillna(method='bfill')  
  
df['GDP_Growth_fill_mean'] =  
df['GDP_Growth'].fillna(df['GDP_Growth'].mean()  
( ))
```

```
df['GDP_Growth_fill_mode'] =
df['GDP_Growth'].fillna(df['GDP_Growth'].mode()[0])
```

5.4 การจัดการค่าผิดปกติ

ค ' า ผิ ด ป ก ต ี (O u t l i e r s) คือจุดข้อมูลที่เบี่ยงเบนไปจากการกระจายทั่วไปของชุดข้อมูลอย่างมีนัยสำคัญ การตรวจจับค่าผิดปกติเป็นขั้นตอนสำคัญในการเตรียมข้อมูล เพราะค่าผิดปกติสามารถบิดเบือนการวัดทางสถิติและนำไปสู่ข้อมูลเชิงลึกที่ไม่ถูกต้องได้

แนวทางการตรวจจับค่าผิดปกติมี 2 วิธีหลัก คือ การใช้ IQR (Interquartile Range) และการใช้สถิติ Z-Score วิธีขั้นสูงเพิ่มเติมจะครอบคลุมในบทที่ 10

5.4.1 แบบฝึกหัด – การตรวจจับค่าผิดปกติด้วย IQR

$IQR = Q3 - Q1$ โดยกำหนดขอบเขตล่างเป็น $Q1 - 1.5 \cdot IQR$ และขอบเขตบนเป็น $Q3 + 1.5 \cdot IQR$ ข้อมูลที่อยู่นอกขอบเขตเหล่านี้ถือว่าเป็นค่าผิดปกติ Box Plot จะวาดขอบเขตเหล่านี้โดยอัตโนมัติ

5.6.1.1 การตั้งค่า

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('/content/Nov2Temp.csv')
```

ชุดข้อมูลอุณหภูมิเดือนพฤศจิกายน มี 118 รายการ 2 คอลัมน์ ได้แก่ high (สูงสุด) และ low (ต่ำสุด)

5.4.1.2 ตรวจสอบค่าผิดปกติ

```
df['low'].hist()
plt.boxplot(df['low'])
```

5.4.1.3 กำหนดขอบเขต

```
low_IQR      = df['low'].quantile(0.75) -  
df['low'].quantile(0.25)  
low_low_limit = df['low'].quantile(0.25) - 1.5 *  
low_IQR # = 4.875  
low_high_limit = df['low'].quantile(0.75) + 1.5  
* low_IQR # = 55.875  
df[df['low'] < low_low_limit] # แสดงค่าผิดปกติด้านล่าง (5  
รายการ)  
df[df['low'] > low_high_limit] # แสดงค่าผิดปกติด้านบน (ไม่มี)
```

5.4.1.4 ลบค่าผิดปกติ

```
df.drop(df[df['low'] < low_low_limit].index,  
inplace=True)  
df.drop(df[df['low'] > low_high_limit].index,  
inplace=True)
```

5.4.1.5 ตรวจสอบผลลัพธ์

```
df['low'].hist()  
plt.boxplot(df['low'])
```

เปรียบเทียบ Histogram และ Box Plot ก่อนและหลังลบค่าผิดปกติ จะเห็นว่าการกระจายข้อมูลสมมาตรและสมดุลมากขึ้น

5.4.1.6 แบบฝึกหัด

ทดลองทำขั้นตอนเดียวกันสำหรับคอลัมน์ df['high']

5.4.2 แบบฝึกหัด – การตรวจจับค่าผิดปกติด้วยสถิติ

ในทางสถิติ ถ้าการกระจายของข้อมูลเป็นแบบ Normal Distribution สามารถใช้ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานเพื่อประมาณความน่าจะเป็นได้ดังนี้:

17. 68% ของข้อมูลอยู่ในช่วง $\text{mean} \pm 1$ ส่วนเบี่ยงเบนมาตรฐาน
18. 95% ของข้อมูลอยู่ในช่วง $\text{mean} \pm 2$ ส่วนเบี่ยงเบนมาตรฐาน

19. 99.7% ของข้อมูลอยู่ในช่วง $\text{mean} \pm 3$ ส่วนเบี่ยงเบนมาตรฐาน ดังนั้นข้อมูลที่อยู่นอก $\text{mean} \pm 3$ ส่วนเบี่ยงเบนมาตรฐานถือว่าเป็นค่าผิดปกติ

5.6.2.1 การตั้งค่า

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/Nov2Temp.csv')
```

5.4.2.2 ตรวจจับค่าผิดปกติ

```
df[(df['low'] < (df['low'].mean() - 3 *
df['low'].std())) |
(df['low'] > (df['low'].mean() + 3 *
df['low'].std()))]
```

ผลลัพธ์: พบ 3 รายการที่เป็นค่าผิดปกติ (แถว 109, 110, 111 มีค่า low = -11, -21, -33)

5.4.2.3 ลบค่าผิดปกติ

```
df.drop((df[(df['low'] < (df['low'].mean() - 3 *
df['low'].std())) |
(df['low'] > (df['low'].mean() + 3 *
df['low'].std()))]).index, inplace=True)
df['low'].plot(kind='box')
```

5.4.2.4 แบบฝึกหัด

ทดลองทำขั้นตอนเดียวกันสำหรับคอลัมน์ df['high']

5.5 การลดขนาดข้อมูล

การลดขนาดข้อมูล (Data Reduction) เป็นเทคนิคสำคัญสำหรับการจัดการกับชุดข้อมูลขนาดใหญ่และซับซ้อน ประกอบด้วย 2 แนวทางหลัก:

20. การลดมิติ (Dimensionality Reduction): ลดจำนวน Feature หรือตัวแปรในขณะที่ยังคงข้อมูลสำคัญไว้ ช่วยปรับปรุงประสิทธิภาพการคำนวณ ลด Noise และเพิ่มความสามารถในการตีความข้อมูล เทคนิคขั้นสูงเช่น PCA และ Feature Selection จะครอบคลุมในบทที่ 8
21. การ Sampling ข้อมูล: เลือกชุดย่อยของจุดข้อมูลจากชุดข้อมูลขนาดใหญ่ เพื่อลดขนาดโดยรวมในขณะที่ยังคงลักษณะและรูปแบบทางสถิติไว้

5.5.1 แบบฝึกหัด – การตัดมิติ (Dimension Elimination)

บางครั้งข้อมูลที่เราเก็บรวบรวมมีข้อมูลมากเกินไปจนความจำเป็นคุณลักษณะบางอย่างอาจไม่มีส่วนร่วมในการวิเคราะห์ของเรา จึงจำเป็นต้องตัดมิติเพื่อโฟกัสที่คุณลักษณะที่ต้องการ

5.7.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

ชุดข้อมูลที่อยู่อาศัยในรัฐแคลิฟอร์เนีย มี 17,000 รายการ และ 9 คอลัมน์

5.5.1.2 การตัดมิติ

มีหลายวิธีในการเลือกหรือตัดคอลัมน์ที่ต้องการ:

```
df_sample1 = df[df.columns[2:]] # เลือกคอลัมน์ตั้งแต่ที่ 3 เป็นต้นไป

df_sample2 = df.drop(df.columns[:2], axis=1) # ลบ 2 คอลัมน์แรก

needed_cols = ['total_rooms', 'total_bedrooms', 'population', 'households']

df_sample3 = df[needed_cols] # เลือกเฉพาะคอลัมน์ที่ต้องการ
```

```
dontneeded_cols =  
['latitude', 'longitude', 'median_income', 'median_  
_house_value']  
df_sample4 = df.drop(dontneeded_cols, axis=1) #  
ลบคอลัมน์ที่ไม่ต้องการ
```

5.5.2 แบบฝึกหัด – การ Sampling

การ Sampling คือการลดจำนวนแถว (Records) ในขณะที่ Dimension Elimination ลดจำนวนคอลัมน์ (Attributes)

5.7.2.1 การตั้งค่า

```
import numpy as np  
import pandas as pd  
df =  
pd.read_csv('/content/sample_data/california_ho  
using_train.csv')
```

5.5.2.2 การ Sampling ตามจำนวน

```
df.sample(n=5) # สุ่ม 5 แถว
```

5.5.2.3 การ Sampling ตามสัดส่วน

```
df.sample(frac=0.001) # สุ่ม 0.1% ของข้อมูล  
df.loc[:10].sample(frac=0.9) # สุ่ม 90% จาก 10 แถวแรก
```

5.5.2.4 การ Sampling แบบมีการทดแทน (With Replacement)

การ Sampling แบบมีการทดแทนช่วยให้สามารถเลือกการเดิมได้มากกว่าหนึ่งครั้ง
เหมาะสำหรับการ Bootstrap Sampling:

```
df.loc[:10].sample(15, replace=True) # สุ่ม 15 แถวจาก  
10 แถวแรก (มีซ้ำได้)
```

5.6 การแยกช่วงและการปรับสเกลข้อมูล

การเตรียมและแปลงข้อมูลให้เหมาะสมกับการวิเคราะห์เป็นขั้นตอนสำคัญ ซึ่งประกอบด้วย 2 แนวคิดหลัก:

22. Data Scaling: กระบวนการแปลงข้อมูลให้อยู่ในช่วงที่สม่ำเสมอ เพื่อให้มั่นใจว่าไม่มี Feature ใด Feature หนึ่งมีอิทธิพลเกินควร ครอบคลุม Min-Max Scaling, Z-Score Standardization และ Decimal Scaling
23. Data Discretization: การแปลงข้อมูลต่อเนื่องเป็นช่วงหรือหมวดหมู่แบบไม่ต่อเนื่อง ช่วยลดความซับซ้อน ลด Noise และทำให้ข้อมูลตีความได้ง่ายขึ้น ครอบคลุม Equal Width Binning และ Equal Frequency Binning

5.6.1 แบบฝึกหัด – การแยกช่วงข้อมูล (Data Discretization)

การ Discretization คือการแปลงคุณลักษณะต่อเนื่องให้เป็นหลายช่วง เพื่อลดความแปรปรวนของข้อมูล

5.6.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

5.6.1.2 แยกช่วง Population

แบ่ง population ออกเป็น 2 กลุ่มโดยใช้ค่าเฉลี่ยเป็นจุดตัด:

```
df['popular'] = np.select(
    [df['population'] < 1429.573941,
     df['population'] >= 1429.573941],
    ['not popular', 'popular'])
df['popular'].value_counts() # not popular: 10,862 popular: 6,138
```


5.6.1.3 แยกช่วง Rooms

แบ่งตามเงื่อนไขหลายข้อโดยใช้ Q1 และ Q3 เป็นเกณฑ์:

```
conditions = [
    (df['total_rooms'] < 1462) &
    (df['total_bedrooms'] < 297), # LL
    (df['total_rooms'] > 3151) &
    (df['total_bedrooms'] > 648), # HH
    (df['total_rooms'] < 2127) &
    (df['total_bedrooms'] > 434), # LH
    (df['total_rooms'] > 2127) &
    (df['total_bedrooms'] < 434), # HL
]
values = ['LL', 'HH', 'LH', 'HL']
df['rooms'] = np.select(conditions, values)
```

5.6.1.4 แยกช่วงมูลค่าบ้าน

สร้างฟังก์ชันแบบกำหนดเองเพื่อแบ่งมูลค่าบ้านเป็น Low, Medium, High:

```
def house_value(value):
    if value < 119400: return 'Low'
    elif value > 265000: return 'High'
    else: return 'Medium'

df['house_value_category'] =
df['median_house_value'].apply(house_value)

df['house_value_category'].value_counts() #
Medium: 8,510 High: 4,247 Low: 4,243
```

5.6.2 แบบฝึกหัด – การปรับสเกลข้อมูล (Data Scaling)

Robust Scaler

$$x' = \frac{x - \text{median}(X)}{IQR(X)}$$

โดยที่:

x' = ค่าที่ผ่านการปรับมาตรฐานแล้ว

x = ค่าข้อมูลเดิม

$\text{median}(X)$ = ค่ามัธยฐานของคุณลักษณะ X

$\text{IQR} = \text{Interquartile Range} = Q3 - Q1$ (ช่วงระหว่างควอร์ไทล์ที่ 3 และ 1)

ข้อดี: Robust Scaler ไม่ได้รับผลกระทบจากค่าผิดปกติ (Outlier) ต่างจาก Min-Max Scaler และ Z-score ที่ไวต่อค่าผิดปกติมาก

5.6.2.1 การตั้งค่า

```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

5.6.2.2 Min-Max Scaling (Normalization)

การแปลงข้อมูลเชิงเส้นให้อยู่ในช่วง $[0, 1]$ โดยใช้สูตร: $v' = (v - \min) / (\max - \min)$

```
df['population_MinMax'] = ((df['population'] -
                             df['population'].min())
                             /
                             (df['population'].max() -
                              df['population'].min()))
```

ผลลัพธ์: ค่า $\min = 0.0$, $\max = 1.0$, $\text{mean} = 0.040$ แสดงว่าข้อมูลถูกปรับให้อยู่ในช่วง 0-1 แล้ว

5.6.2.3 Z-Score Standardization

การปรับข้อมูลโดยใช้ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน โดยใช้สูตร: $v' = (v - \text{mean}) / \text{std}$ ผลลัพธ์จะมีค่าเฉลี่ย = 0 และส่วนเบี่ยงเบนมาตรฐาน = 1

```
df['population_Z'] = ((df['population'] -
                        df['population'].mean())
```

```
/ df['population'].std())
```

ผลลัพธ์: $\text{mean} \approx 0$, $\text{std} = 1.0$ แสดงว่าข้อมูลถูกทำให้เป็นมาตรฐานแล้ว

5.6.2.4 Decimal Scaling

การปรับสเกลโดยการเลื่อนจุดทศนิยม โดยหารด้วย 10^t เมื่อ t = จำนวนหลักของค่าสูงสุด + 1 ตัวอย่าง: ค่าสูงสุดของ population = 35,682 (5 หลัก) ดังนั้น $t = 5$, หารด้วย 100,000:

```
df['population_decimal'] = df['population'] / 100000
```

ผลลัพธ์: $\text{min} = 0.00003$, $\text{max} = 0.35682$ ซึ่งง่ายต่อการตีความ

เปรียบเทียบทั้ง 3 วิธีพร้อมกัน:

```
df[['population', 'population_MinMax', 'population_Z', 'population_decimal']].describe()
```

5.7 คลังข้อมูล (Data Warehouse)

คลังข้อมูล (Data Warehouse) คือระบบจัดเก็บข้อมูลส่วนกลางที่ออกแบบมาเพื่อรวบรวมข้อมูลจากหลายแหล่ง เพื่อให้เข้าถึงได้สำหรับการวิเคราะห์ ส่วนประกอบหลักของ Data Warehouse ได้แก่:

24. Data Cube: โครงสร้างข้อมูลหลายมิติที่ช่วยให้จัดเก็บและวิเคราะห์ข้อมูลในมุมมองต่างๆ แต่ละมิติแทนลักษณะหรือคุณลักษณะของข้อมูล
25. Pivot Table: เครื่องมือในตัวของ Pandas ที่ช่วยสรุป วิเคราะห์ และนำเสนอข้อมูลในรูปแบบไดนามิก

5.7.1 แบบฝึกหัด – Data Cube

5.7.1.1 การตั้งค่า

```
!pip install atoti
import pandas as pd
import atoti as tt
```

5.7.1.2 สร้าง Session และโหลดข้อมูล

```
session = tt.Session()

df = pd.read_csv('/content/Spotify_Youtube_Sample.csv')

views = session.read_csv('/content/Spotify_Youtube_Sample.csv', keys=['Artist', 'Track', 'Album'])
```

5.7.1.3 สร้าง Data Cube

```
cube = session.create_cube(views, name='Views')
m = cube.measures # Measures (ค่าที่วัด)
l = cube.levels # Levels (มิติ)
```

5.7.1.4 สืบค้นข้อมูลจาก Cube

การสืบค้นแบบ 0 มิติ (Apex): ยอดรวม Views ทั้งหมด:

```
cube.query(m['Views.SUM']) # ผลลัพธ์ :
1,902,053,002,307
```

การสืบค้นแบบ 1 มิติ: Views จำแนกตาม Artist หรือ Album:

```
cube.query(m['Views.SUM'],
levels=[l['Artist']])

cube.query(m['Views.SUM'], levels=[l['Album']])
```

การสืบค้นแบบ 2 มิติ: Views จำแนกตาม Album และ Artist พร้อมกัน:

```
cube.query(m['Views.SUM'], levels=[l['Album'],
l['Artist']])
```

การสืบค้นแบบ 2 มิติพร้อม Slicing: กรองข้อมูลสำหรับ Artist เฉพาะราย:

```
cube.query(m['Views.SUM'],
           levels=[l['Album'],l['Track']],
           filter=l['Artist']=='The Beatles')

cube.query(m['Views.SUM'],
           levels=[l['Album'],l['Track']],
           filter=l['official_video']=='True')
```

การสืบค้นแบบ 3 มิติ:

```
cube.query(m['Views.SUM'],
           levels=[l['Album'],l['Track'],l['official_video']])
```

5.7.2 แบบฝึกหัด – Pivot Table

Pivot Table เป็นเครื่องมือสำหรับ OLAP (Online Analytical Processing) ที่ช่วยสรุปและวิเคราะห์ข้อมูลในรูปแบบตารางที่ยืดหยุ่น

5.7.2.1 การตั้งค่า

```
import numpy as np
import pandas as pd

df =
pd.read_csv('/content/Spotify_Youtube_Sample.csv')
```

5.7.2.2 สร้าง Pivot Table

```
df_pivot = df.pivot_table(
    values=['Views','Likes','Comments'],
    index=['Artist','Album'],
    aggfunc='mean')
```

ผลลัพธ์: Pivot Table แสดงค่าเฉลี่ย Views, Likes และ Comments จำแนกตาม Artist และ Album (13,955 แถว)

5.7.2.3 สืบค้นข้อมูลจาก Pivot Table

```
df_pivot.loc['Michael Jackson'] # ข้อมูลของ Michael
Jackson
df_pivot.loc[['Michael Jackson', 'The
Beatles', 'Beyonce']] # หลาย Artist
```

5.7.2.4 เรียงลำดับข้อมูล

```
df_pivot.sort_values(by='Views',
ascending=False) # เรียงตาม Views มากไปน้อย
```

ผลลัพธ์: Daddy Yankee (VIDA) มียอดชมสูงสุด 8,079,647,000 ครั้ง ตามด้วย Charlie Puth และ Wiz Khalifa (See You Again)

```
df_pivot.sort_values(by=['Artist', 'Views'],
ascending=False) # เรียงตาม Artist แล้วตาม Views
df_pivot.sort_values(by=['Artist', 'Views'],
ascending=False).loc['Daddy Yankee']
df_pivot.sort_values(by=['Artist', 'Views'],
ascending=False).loc[['Coldplay', 'The
Beatles']]
```

ตัวอย่างการประยุกต์ใช้งานจากงานวิจัย

ผาพันธ์และคณะ (2025) ในงานวิจัยเรื่อง "Detecting Automobile Insurance Fraud: A Novel Two-Step Strategy Using Effective Ensemble Learning Techniques" ได้นำเทคนิคการเตรียมข้อมูลมาใช้อย่างเป็นระบบ โดยใช้ Under-sampling และ SMOTE (Synthetic Minority Over-sampling Technique) เพื่อแก้ปัญหา ข้อมูลไม่สมดุล (Imbalanced Data) ซึ่งสัดส่วนของการฉ้อโกงมีน้อยกว่า การเรียกร่องปกติมาก และใช้ Mutual Information เป็นเครื่องมือในการ คัดเลือกตัวแปร (Feature Selection) เพื่อเลือกเฉพาะตัวแปรที่มีความสำคัญ ก่อนนำไปสร้างแบบจำลอง

ในทำนองเดียวกัน สิมมาจันท์และคณะ (2023) ในงานวิจัยเรื่อง "Detecting Fraudulent Claims in Automobile Insurance Policies by Data Mining Techniques" ได้ใช้

Logistic Regression และ Random Forest เป็นเครื่องมือในการ คัดเลือกตัวแปร (Variable Selection) พร้อมทั้งใช้ Cross-validation เพื่อลดความไม่แน่นอน (Uncertainty) ของแบบจำลอง ซึ่งกระบวนการเหล่านี้ล้วนเป็นส่วนหนึ่งของขั้นตอนการเตรียมข้อมูลที่กำลังกล่าวถึงในบทนี้

นอกจากนี้ วงศ์วิวัฒน์และคณะ (2025) ในงานวิจัยเรื่อง "Machine Learning Approaches for Credit Risk Assessment in SMEs" ได้ใช้ K-fold Cross-validation เพื่อประเมินประสิทธิภาพแบบจำลองและทำการ Balance ข้อมูลก่อนการสร้างแบบจำลอง ซึ่งแสดงให้เห็นว่า การเตรียมข้อมูลที่ดี ส่งผลโดยตรงต่อความแม่นยำของแบบจำลองเหมืองข้อมูล

เอกสารอ้างอิง

1. Simmachan, T., Manopa, W., Neamhom, P., Poothong, A., & Phaphan, W. (2023). Detecting fraudulent claims in automobile insurance policies by data mining techniques. *Thailand Statistician*, 21(3), 552–568.
2. Phaphan, W., Sathitvudh, S., Suntornsuwan, T., Budsaba, K., & Simmachan, T. (2025). Detecting automobile insurance fraud: A novel two-step strategy using effective ensemble learning techniques. *Thailand Statistician*, 23(1), 144–161.
3. Wongwiwat, P., Chairojwattana, A., & Phaphan, W. (2025). Machine Learning approaches for credit risk assessment in SMEs. *Thailand Statistician*, 23(3), 710–731.